Prokoti

Collaborative PDF Viewer — Technical Overview

Architecture · Libraries · Design Decisions

Wed May 13 2026

This document describes the complete technology stack behind Prokoti — a browser-based collaborative PDF viewer. It explains why each library was chosen over alternatives, how the PDF rendering pipeline works, and how the real-time annotation and e-signature systems are implemented end to end.

Next.js 14 NestJS PostgreSQL pdfjs-dist 4 Socket.IO AWS S3

Contents

1. Project Architecture . . . 3
2. Technology Stack . . . 3
3. PDF Rendering Pipeline . . . 5
4. Text Layer & Text Selection . . . 6
5. Annotation System . . . 7
6. Real-Time Sync (Socket.IO) . . . 9
7. E-Signature System . . . 10
8. Authentication & Security . . . 11
9. Document Storage (AWS S3) . . . 12
10. Watermark Injection . . . 12

1. Project Architecture

Prokoti is a monorepo structured into three packages: a Next.js 14 frontend, a NestJS backend, and a shared TypeScript types package. The three apps communicate via a REST API and a Socket.IO WebSocket connection. Documents are stored on AWS S3; metadata and annotations live in PostgreSQL managed through Prisma ORM.

Monorepo Layout

```
/apps/frontend   — Next.js 14 (App Router, TypeScript, Tailwind CSS)
/apps/backend    — NestJS + Prisma + Socket.IO gateway
/packages/types  — Shared DTOs (AnnotationDto, DocumentDto, …)
docker-compose.yml  PostgreSQL 15 container
```

The frontend is a pure SPA-style page inside the Next.js App Router. The PDF viewer renders entirely in the browser using pdfjs-dist; the backend never re-renders pages. Annotations are overlaid as absolute-positioned HTML elements on top of the canvas, keeping the PDF layer and the interaction layer cleanly separated.

2. Technology Stack

Frontend

Next.js 14	App Router	React framework — server components, file-based routing, middleware
TypeScript	5.x	Static typing shared via /packages/types
Tailwind CSS	3.x	Utility-first styling, zero runtime CSS-in-JS overhead
pdfjs-dist	4.10.38	Mozilla PDF rendering engine — canvas + text layer
signature_pad	4.x	Smooth freehand e-signature drawing on <canvas>
Socket.IO	client 4.x	Real-time annotation sync over WebSocket

Backend

NestJS	10.x	Modular Node.js framework — DI, Guards, Decorators
Prisma ORM	5.x	Type-safe database access with migration tooling
PostgreSQL	15	Primary data store for users, documents, annotations
Socket.IO	server 4.x	WebSocket gateway for broadcast events

AWS SDK v3	@aws-sdk	S3 client — PutObject / GetObject (backend-only)
pdf-lib	1.17	In-memory PDF manipulation for watermark injection
@nestjs/throttler	5.x	Rate-limiting on auth endpoints (5 req/min)
xss	1.x	XSS sanitization for annotation content
bcrypt	5.x	Refresh-token hashing before DB storage
passport-jwt	4.x	JWT strategy — reads httpOnly cookie

3. PDF Rendering Pipeline

The PDF viewer uses pdfjs-dist 4.x, Mozilla's JavaScript port of the PDF rendering engine. Each page is rendered independently into an HTML5 <canvas> element; a transparent text layer is overlaid on top to enable text selection and search.

Why pdfjs-dist instead of react-pdf or react-pdf-viewer?

- Full access to the low-level pdfjs API: viewport, text content, operator lists.
- react-pdf wraps pdfjs but hides the TextLayer API, making custom overlays awkward.
- react-pdf-viewer adds a large bundle and opinionated UI that conflicts with our custom toolbar.
- Direct use of pdfjs lets us co-locate canvas and text layer in one React component with precise control.

Rendering Steps (per page)

1. **Load document** — pdfjs.getDocument(url) returns a PDFDocumentProxy. The worker runs in a separate thread via /pdf.worker.min.mjs served from the Next.js public/ folder.
2. **Build viewport** — page.getViewport({ scale }) computes the pixel-space transform. scale = user zoom × devicePixelRatio for crisp rendering on HiDPI screens.
3. **Size canvas** — canvas.width/height = viewport.width/height. CSS width/height = viewport.width/height / devicePixelRatio so the element fits the page without blur.
4. **Render to canvas** — page.render({ canvasContext, viewport }).promise draws all PDF operators (text, images, paths) onto the canvas in the pdfjs worker thread.
5. **Build text layer** — page.getTextContent() returns spans with bounding boxes in PDF coordinates. new pdfjs.TextLayer({ textContentSource, container, viewport }) maps them to CSS absolute positions.
6. **setLayerDimensions** — pdfjs.setLayerDimensions(div, viewport) writes the CSS --scale-factor custom property. The pdfjs text layer stylesheet uses this variable to scale font sizes correctly.
7. **Font ready** — await document.fonts.ready ensures PDF-embedded fonts have been parsed before the text layer renders, preventing invisible-text frames.

Worker file location

The pdfjs worker (pdf.worker.min.mjs, ~1.4 MB) is copied into /apps/frontend/public/ so Next.js serves it as a static asset. This avoids CDN availability issues and works in offline/intranet environments.

4. Text Layer & Text Selection

The text layer renders invisible but selectable elements on top of the canvas. It is the bridge between the visual PDF and user interactions (selection, search, annotation).

pdfjs 4.x API change

In pdfjs-dist 3.x the API was renderTextLayer(params). This function was removed in v4.x. The replacement is the TextLayer class:

Old API (v3 — removed)

```
import { renderTextLayer } from "pdfjs-dist";
awaitPage renderTextLayer({ textContentSource, container, viewport }).promise;
```

New API (v4+)

```
const tl = new pdfjs.TextLayer({ textContentSource: textContent, container: div, viewport });
await tl.render();
```

Coordinate System

PDF coordinates have the origin at the bottom-left corner and y increases upward. The viewport transform flips this so the canvas origin is top-left with y increasing downward, matching the browser DOM. All annotation rects are stored in canvas-space (post-viewport) coordinates at scale=1, then multiplied by the current scale at render time.

Annotation Overlay Alignment

- The canvas, text layer , and annotation overlay are siblings inside a position:relative wrapper.
  - All three share the same pixel dimensions (viewport.width × viewport.height at scale=1).
  - Annotation rects stored as {x, y, w, h} in PDF canvas units; rendered as left: x*scale, top: y*scale.
    - This means the overlay stays perfectly aligned regardless of the user's zoom level.
    -

5. Annotation System

Prokoti supports five annotation types. All are stored in PostgreSQL and synced in real time to every connected client viewing the same document.

Annotation Types

HIGHLIGHT
Coloured semi-transparent rectangle over selected text. Multiple rects per annotation for multi-line selections.

UNDERLINE
Horizontal line under selected text (border-bottom, no fill). Uses the same rect array as HIGHLIGHT.

NOTE
Single-point sticky note. Stores a rect with the anchor position plus free-text content.

DISCUSSION
Like NOTE but supports threaded replies (DiscussionReply table) and sharing with named members.

SIGNATURE
Base-64 encoded PNG of the drawn signature. Stored as a single rect that tracks position and size; owner can drag/resize.

Data Flow: Creating a Highlight

1. User drags on the text layer !' mouseup fires in PDFViewer.
2. window.getSelection() extracts the selected text and bounding rects.
3. Rects are converted from viewport pixels back to scale=1 coordinates.
4. onAnnotationCreate(dto) is called; the hook POSTs to POST /annotations.
5. Backend validates, sanitizes content (xss.filterXSS), persists to PostgreSQL.
6. Backend emits annotation:created to all sockets in the doc:{documentId} room.
7. useAnnotations hook updates local state optimistically — no wait for the socket echo.
8. AnnotationOverlay renders the new highlight div immediately.

Visibility & Privacy

- isPrivate: true — only the creator sees the annotation (filtered client-side and server-side).
- isPrivate: false — all document viewers can see it.
- share(annotationId, memberIds) adds rows to DiscussionMember and sets isPrivate: false.
- The /annotations/document/:id endpoint applies an OR filter: own annotations "* shared annotations.

Rect Storage

PostgreSQL schema

```
rects  Json   -- stored as [{ x: number, y: number, w: number, h: number }, ...]
              -- coordinates are in PDF canvas units at scale=1
              -- multiple rects per annotation for multi-line text selections
```

6. Real-Time Sync (Socket.IO)

Every annotation mutation is broadcast to all other clients viewing the same document. Socket.IO was chosen over polling and raw WebSockets because it provides rooms, automatic reconnection, and a familiar event API.

Why Socket.IO instead of polling or raw WebSocket?

- Rooms: socket.join("doc:xyz") lets the gateway broadcast only to viewers of that document.
- Automatic reconnect with exponential back-off — no user-visible interruption on network blip.
- Falls back to HTTP long-poll in environments that block WebSocket upgrades.
- NestJS has first-class @WebSocketGateway support with full dependency injection.
- Polling adds latency and server load proportional to the number of active sessions.

Gateway Architecture

The NestJS gateway (AnnotationsGateway) handles authentication by reading the access_token httpOnly cookie from the socket handshake headers. On connection it joins the client to the room matching the documentId provided in the join-document event.

Event Reference

join-document { documentId } !' client joins room doc:{documentId}
leave-document { documentId } !' client leaves room
annotation:created { annotation } !' new annotation persisted
annotation:updated { annotation } !' content / rects changed
annotation:deleted { annotationId } !' annotation removed
reply:created { reply, annotationId } !' new reply in discussion

Optimistic Updates

The frontend does not wait for the socket echo to update its own UI. After a successful API call the useAnnotations hook applies the change to local state immediately. If the socket event arrives later (e.g., from another tab), the deduplication check (prev.some(a => a.id === id)) prevents duplicates.

CORS Configuration

Both the HTTP server (main.ts) and the @WebSocketGateway decorator use a function-based CORS origin check that allows any localhost:* origin. This is necessary because Next.js dev server can bind to a different port than the default (3000 vs 3002 etc.).

7. E-Signature System

The signature feature lets users draw a freehand signature on a canvas and place it anywhere on any page of the document. The full pipeline spans the SignaturePickerModal, the AnnotationOverlay, and the standard annotation REST/WebSocket path.

Why signature_pad?

- Pressure-sensitive B-spline smoothing produces natural-looking strokes.
- Exports directly to PNG data URL — no server round-trip for the initial capture.
- Lightweight (~5 KB gzipped) — no canvas library dependency like fabric.js.
- Active maintenance (GitHub: szimek/signature_pad).
- fabric.js was evaluated but is 10× larger and adds abstraction not needed for a single-use signature canvas.

Two-Step Placement Flow

Draw	SignaturePickerModal renders a <canvas> element wired to a signature_pad instance. User draws; the pad captures pointer events and renders smooth bezier curves.
Export	On "Use Signature" the pad calls toDataURL("image/png") to produce a base-64 encoded PNG string (~5–30 KB).
Pending	The data URL is stored as pendingSignature state in the viewer page. The cursor changes to crosshair and the toolbar shows "Cancel placement".
Place	User clicks anywhere on the PDF canvas. The click handler captures the (x, y) position in scale=1 coordinates and emits onAnnotationCreate with type=SIGNATURE, rects=[{x,y,w:120,h:60}], content=dataUrl.
Persist	Backend stores the annotation like any other. The content column holds the full data URL (up to ~500 KB for a detailed signature).
Overlay	AnnotationOverlay renders a <img src={annotation.content}> inside a ResizableSignature component. The owner can drag the image or drag corner handles to resize.

Drag & Resize

The ResizableSignature component in AnnotationOverlay.tsx implements its own drag system using window-level mousemove/mouseup listeners (attached on mousedown, removed on mouseup). Corner handles allow proportional resizing. On mouseup, the updated rect is sent to PATCH /annotations/:id, persisted, and broadcast to other viewers via annotation:updated.

8. Authentication & Security

JWT Dual-Token Strategy

Prokoti uses two JWTs: a short-lived access token and a long-lived refresh token. Both are stored in httpOnly; SameSite=Strict cookies — they are never accessible to JavaScript, mitigating XSS-based token theft.

Token Lifetimes

Access token: 15 minutes — extracted from req.cookies.access_token by passport-jwt

Refresh token: 7 days — bcrypt hash stored in DB; raw value in cookie only

Rotation: Each use of the refresh token issues a new pair (token rotation)

Security Measures

- Rate limiting: @nestjs/throttler limits /auth/login and /auth/register to 5 requests/min/IP.
- XSS sanitization: all user-supplied text (annotation content, selectedText, replies) passes through xss.filterXSS() before DB writes.
- Refresh token hashing: the raw refresh token is never stored; bcrypt.compare() validates it on each refresh.
- CORS: function-based origin check; credentials: true; only localhost origins allowed in development.
- S3 credentials: backend-only, never exposed to the frontend. The frontend calls the backend API which proxies S3 responses.
- Magic byte validation: document uploads check the first bytes of the file, not just the MIME type.

Auth Flow Summary

1. POST /auth/register — hash password with bcrypt, create User, issue access + refresh tokens as cookies.
2. POST /auth/login — validate credentials, issue fresh token pair.
3. Every protected route — JwtAuthGuard reads access_token cookie, verifies signature, attaches user to req.
4. POST /auth/refresh — JwtRefreshGuard reads refresh_token cookie, looks up DB hash, issues new pair.
5. POST /auth/logout — clears both cookies and nullifies the stored refresh hash.

9. Document Storage (AWS S3)

Documents are stored as objects in an S3 bucket. The backend uses @aws-sdk/client-s3 v3. S3 credentials are kept in the backend .env file and are never sent to the browser.

Upload Flow

- Frontend POSTs the file to POST /documents as multipart/form-data.
- Backend validates magic bytes (PDF: %PDF header, DOCX/PPTX: PK zip header).
- Non-PDF files are converted to PDF via libreoffice-convert before upload.
- Backend calls PutObjectCommand with the PDF buffer; stores the S3 key in PostgreSQL.
- The frontend receives the DocumentDto (no S3 URL, no credentials).

Download / Print Flow

- Frontend requests GET /documents/:id/download (or /print) from the backend.
- Backend calls GetObjectCommand to stream the original PDF from S3.
- pdf-lib injects a per-user watermark in memory (not written back to S3).
- Backend streams the watermarked PDF buffer to the client with the correct Content-Disposition header.

10. Watermark Injection

When a user downloads or prints a document, the backend injects their name as a diagonal watermark on every page. The original file in S3 is never modified.

pdf-lib watermark implementation

1. Fetch S3 object !' Buffer
2. PDFDocument.load(buffer)
3. For each page:
   page.drawText(username, { x: w/2, y: h/2, size: 48, opacity: 0.3,
   rotate: degrees(35), color: rgb(0.5,0.5,0.5) })
4. doc.save() !' new Buffer (never touches S3)
5. Stream to HTTP response

Why pdf-lib instead of pdfkit or Ghostscript?

- pdf-lib can load an existing PDF and modify it; pdfkit only creates new PDFs from scratch.
- No native binary dependency (unlike Ghostscript or Puppeteer) — runs in a standard Node.js process.
- Pure JavaScript — works on Windows, Linux, and macOS without additional system packages.
- In-memory operation — watermark is injected in the request handler with no temp files.

Prokoti — Built with open-source tooling

Next.js · NestJS · Prisma · PostgreSQL · pdfjs-dist · Socket.IO · AWS S3 · pdf-lib · signature_pad